

Reviewer checklist

Handout. Week 1, Day 2. Keep this. You will use it for the next six weeks.

Before you read a single line

```
python3 -m ruff check .  
python3 -m mypy src tests  
python3 -m pytest -q
```

If any of the three is red, **stop and send it back**. You are not a linter. Every minute spent by eye on something a tool reports in forty milliseconds is a minute not spent on the thing only you can catch.

Then read in this order, not the order the diff gives you

1. **The commit messages.** `git log --oneline main..<branch>`
Does each commit do one thing? Does the message say what and why?
2. **The file list.** `git show --stat <sha>` or `git diff --stat main..<branch>`
Is every file in this change part of this change? Anything unexpected here is the most likely place a problem is hiding, because nobody looks.
3. **The tests.** Before the implementation.
What behaviour do they claim? Would they fail if the code were wrong?
4. **The implementation.** Now, and only now.
5. **What is missing.** The hardest read. A diff cannot show you the test nobody wrote.

The five questions

- **Does it do what the ticket says?** Read the ticket, then read the code. Boundaries and the word "and above" are where rules go wrong.
- **Would the tests fail if the code were wrong?** Break the function in your head. Does anything notice?
- **What is not here?** The missing test, the unhandled case, the untouched caller.

- What happens on the unhappy path? Empty input, absent value, wrong type, zero.
- Is every file in this diff part of this change?

Write findings like this

Field	What goes in it
Where	File and line
What	One sentence
Class	MACHINE if a gate should have caught it, HUMAN if only a person could
Ask	What you want the author to do

Then a verdict: **approve**, **approve with comments**, or **request changes**, and one sentence saying why.

How to say it

- **Ask, do not instruct**, where you are not certain. "What happens at exactly 500.00?" finds the bug and leaves the author their reasoning. "Change this to `>=`" gets the line changed and teaches nothing.
- **Separate the blocking from the optional**. If everything reads as equally urgent, the author will guess, and they will guess wrong.
- **Review the code, not the person**. "This function returns None on an unknown band", not "you forgot".
- **Do not rewrite it**. A review that arrives as a patch has taken the decision away.

A review is a decision, not a list

If you found something that makes the code wrong and you approved anyway, you did not review it. You commented on it. The question is always: **would you put your name next to this?**